Individual Reflection: Floyd-Rivest (Ben Hood)

1. Worst case and what triggers it

Floyd-Rivest has no worst-case guarantee, so in theory it can be pushed to $O(n^2)$. In practice, its weak point is that it takes its "sample" from consecutive positions around k, which only works if position says nothing about value. Any input where position and value are related can mislead it. On organ pipe input, where the median's window sits at the peak, it needed 9.3 times its random-input comparisons at $n = 10^7$. Against McIlroy's adversary, which picks every comparison outcome so the window keeps producing bad pivots, it reached 106 times its normal comparisons at $n = 10^5$ (185 per element). That ratio grew steadily with n, from 17 times at 10^3, so the cost is superlinear. It stayed far below quickselect's roughly 18,750 per element, though.

2. Hardest bug or design decision

The hardest design decision was whether to implement the algorithm as published, with its window taken from consecutive positions around k, or to sample the window at random. Random sampling is what the probabilistic analysis assumes, and it would have protected against structured inputs. But the point of the project was to test the algorithm people actually use and cite, and the contiguous window is also what makes it cheap as there's no random number generation and no extra swaps. I kept the original as our benchmarked version and treated random sampling as a separate experiment. That turned out to be the right call, because the contiguous window is responsible for my most interesting results: sorted input became perfect (1.00n), while reverse-sorted (2.56n) and organ pipe (9.3× normal) broke it. Had I randomised from the start, every input would have looked like random input, and I would never have found out how much the algorithm depends on its window.

3. What I would do differently

Despite happily implementing the algorithm as published, I would like to have built the random-window version alongside the original from the start, as an option in the benchmark harness, rather than testing it only at the end. The contiguous window caused both the organ pipe result and my reverse-sorted surprise, but my evidence for the fix is narrow. My trace program randomises only the top-level window, and I ran it on just three cases. It brought reverse-sorted input from 2.56n down to 1.57n, the same as random input, while sorted input rose from 1.00n to 1.53n, because the random window loses the lucky exact pivot. With both versions in the harness from day one, the same sweep would have measured that trade-off on every input family and size, and against the adversary.

4. What I now understand

At first, I thought of Floyd–Rivest as simply "picking a good pivot from a sample". I didn't understand why the sample window is shifted off-centre by sd, or where the expected comparison count of n + min(k, n-k) comes from. Running experiments and tracing the algorithm round by round helped me understand that the shift makes the pivot deliberately overshoot the target, so the target ends up near the edge of the remaining range and the next partition has much less work to do. For the median, this means roughly one pass over n elements followed by another over n/2, giving approximately 1.5n comparisons. For the minimum or maximum, it is closer to n. I tested this by setting sd to zero. Comparisons increased from 1.29n to 1.85n at the 25th percentile, and from 1.57n to 2.08n at the median. Median runs varied between 1.6n and 2.8n depending on which side the pivot landed. This also explained my reverse-sorted result: in a reversed window the order flips the shift's direction, so the pivot lands just short of the target every round instead of just past it.